

CloudRouteAI: Comprehensive Technical Updates Report

Date: May 10, 2026 **Subject:** 5-Page Technical Status & Update Report for CloudRouteAI Platform

1. Executive Summary

CloudRouteAI is an advanced network simulation platform designed to bridge the gap between low-level packet simulation and high-level artificial intelligence. Over the course of the project lifecycle, significant technical milestones have been achieved to transform a basic network simulation into an autonomous, "self-healing," and highly adaptive routing system.

This report outlines all the technical updates, architectural decisions, and module integrations completed thus far. The platform now successfully integrates the NS-3 simulation engine, a Machine Learning (ML) inference pipeline, a dynamic Threshold-based Dijkstra routing controller, and a premium interactive evaluation dashboard.

2. Phase 1 & 2 Technical Updates: NS-3 Simulation & Telemetry Engine

The foundation of the project required a robust environment to simulate realistic network behaviors, including normal traffic flow, link failures, and congestion events.

2.1 Network Topology Design & Implementation

A 10-node complex network topology was implemented directly in C++ using the NS-3 API.

- **Node Configuration:** Nodes were configured with point-to-point links.
- **Routing Paths:** The topology was deliberately designed with a forced choice:
 - **Primary Path:** 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 (Shorter, lower hop count)
 - **Alternate Path:** 2 -> 8 -> 9 -> 4 (Longer, higher hop count detour)
- **Technical Goal:** By making the primary path shorter, the default static routing always prefers it. This sets the stage to prove that the ML routing engine can mathematically override physical hop-count advantages when congestion or failure occurs.

2.2 Custom C++ Telemetry Extraction via FlowMonitor

Instead of relying entirely on post-simulation packet capture (PCAP) analysis, we implemented a custom runtime monitoring module in C++.

- **Runtime Polling:** The system hooks into NS-3's `FlowMonitor` and extracts real-time statistics every 2 seconds during the simulation runtime.
- **Metrics Tracked:** `queue_utilization` (buffer depth), `delay_ms` (latency), `throughput_mbps`, and `packet_loss_rate`.
- **Data Export:** Telemetry data is exported automatically into JSON formats (`outputs/`

`raw/`), allowing for asynchronous consumption by the Python data processing pipeline.

2.3 Event-Driven Scenario Generation

We hardcoded three primary network disruption scenarios within the C++ simulation environment to test the ML model's responsiveness:

1. **Normal Scenario:** Baseline traffic flow with zero intentional disruptions.
2. **Congestion Scenario:** At $t = 4s$, Link 4→5 is maliciously throttled (bandwidth reduced to 1 Mbps), causing queue backup and latency spikes.
3. **Failure Scenario:** At $t = 8s$, Link 4→5 is programmatically disconnected via NS-3 API state changes, causing 100% localized packet loss.

3. Phase 3 Technical Updates: Machine Learning Prediction Engine

With the telemetry pipeline established, the next milestone was replacing raw physical metrics with an intelligent "Routing Cost" mechanism.

3.1 Synthetic Dataset Generation

To train the model, we generated a synthetic dataset (`ns3_network_flows.csv`) consisting of 1,000 rows and 7 features. This data represents various network states based on specific routing philosophies:

- **Healthy Link:** Low delay, zero loss $\rightarrow$ Assigned Cost ~ 10
- **Congested Link:** High queue utilization, moderate delay $\rightarrow$ Assigned Cost ~ 1000
- **Failed Link:** Disconnected, massive packet drop $\rightarrow$ Assigned Cost 9999

3.2 Model Selection and Training (`train.py`)

We selected a **RandomForestRegressor** from `scikit-learn` due to its ability to handle non-linear relationships between variables (e.g., packet loss and delay).

- **Feature Normalization:** Applied Min-Max normalization across all input features.
- **Model Performance:** The training achieved an exceptional R^2 Score of 0.9997 and a Mean Squared Error (MSE) of 320.37.
- **Artifact:** The trained model is serialized into `model.pkl` for rapid inference during runtime.

3.3 Prediction Pipeline (`predict.py`)

A Python-based inference engine was implemented to digest the real-time `outputs/processed/*.json` metrics.

- The script loads the trained model, parses the incoming telemetry, and predicts the exact routing cost for all paths.
- It writes the outputs to `outputs/ml/*_costs.json` for consumption by the C++ controller.

4. Phase 4 Technical Updates: Dynamic Routing &

Threshold Control

The most critical update involved linking the ML predictions back into the NS-3 simulation to actively alter routing tables on the fly.

4.1 Alpha Threshold Controller ($\alpha = 0.15$)

To prevent "route flapping" (oscillation between paths due to minor cost variations), we implemented a Threshold Controller mechanism.

- The network establishes a baseline cost at $t = 2s$.
- The threshold is set using a multiplier: $\text{baseline} * (1 + 0.15)$.
- The controller actively polls the ML-predicted costs. A reroute is *only* triggered if the ML cost exceeds the threshold by more than 15%.

4.2 Runtime Dijkstra Integration

When the Alpha Threshold is breached, the C++ `AdaptiveRoutingController` activates:

- It constructs a temporary graph using the newly predicted ML costs (where congested links now weigh 1000+).
- It executes Dijkstra's Shortest Path algorithm across this graph.
- It dynamically rewrites the `Ipv4StaticRouting` tables in the active NS-3 nodes, physically diverting traffic to the alternate path (9 → 10).

4.3 Handling Complex Edge Cases (Traffic Spikes)

We successfully integrated logic to prevent blind rerouting. If a massive traffic spike degrades the entire network, Dijkstra evaluates both paths. If the alternate path is worse than the congested primary path, the controller outputs `THRESHOLD_BREACHED_NO_BETTER_PATH` and intelligently remains on the current path, maintaining network stability.

5. Phase 5 Technical Updates: The Explainability Dashboard

Because AI decisions act as a "black box," we developed a comprehensive Streamlit dashboard to serve as an explainability and visualization layer for stakeholders and evaluators.

5.1 Glassmorphism UI & Premium Aesthetics

The dashboard frontend was completely overhauled using modern web design principles:

- Implemented customized CSS rendering a **Glassmorphism** aesthetic (translucent panels, gradient backgrounds, and sleek shadows).
- Configured a multi-tab layout for seamless navigation between the Project Overview, Live Network Visualizations, and Routing Analytics.

5.2 Real-time KPI Tracking & Visualizations

The dashboard reads directly from the `outputs/` directories to render metrics dynamically:

- **Plotly Integration:** Interactive charts for Packet Loss, Network Load, and Throughput comparison over time.
- **Comparative Analysis:** Dedicated side-by-side benchmarking of Static Routing vs. Adaptive (ML) Routing.

5.3 The AI Intelligence Feed

To provide transparency, we implemented a live "Intelligence Feed" (event log). This feed translates the raw data into human-readable logs (e.g., *"Event: ML detected cost spike (Cost: 3105.4). Threshold breached. Routing updated via alternate path."*), making the system's reasoning explicit.

6. End-to-End Automated Pipeline Integration

The entire workflow has been fully automated, ensuring zero manual intervention is required to generate the network, process the data, predict the models, and view the dashboard.

6.1 `run_all.sh` Master Script

We implemented a master bash script to orchestrate the pipeline:

1. Cleans the `outputs/` and `tmp_unzip/` directories to prevent data staleness.
2. Executes the baseline NS-3 C++ simulation to generate initial telemetry.
3. Invokes the Python data parsers to convert XML/raw data into JSON.
4. Triggers `predict.py` to generate the ML routing costs.
5. Reruns the simulation with the adaptive routing flag enabled.
6. Launches the Streamlit dashboard on `localhost:8501`.

7. Validated Results & Performance Benchmarking

The latest technical updates have yielded highly successful validation metrics across our testing scenarios:

- **Congestion Mitigation:** In the throttled link scenario, the ML model correctly detected the anomaly at $t = 8s$. Adaptive routing engaged, recovering throughput from 1 Mbps back to ~ 8 Mbps, effectively eliminating queue bottlenecks on the primary link.
- **Failure Recovery:** In the link failure scenario, the model successfully registered the infinite cost (9999). The network self-healed instantaneously via the alternate path, reducing packet loss from 100% to near zero for the remainder of the simulation.
- **Stability Maintained:** The implementation of the Alpha Threshold successfully prevented route oscillation, proving the system is robust enough for production-level simulation requirements.

8. Conclusion

The technical updates executed up to this point have transformed CloudRouteAI into a sophisticated, end-to-end framework. The successful integration of NS-3 telemetry with a custom-trained RandomForestRegressor, governed by an intelligent threshold-based routing controller, effectively solves the problem of static routing limitations. The addition of the Glassmorphism Streamlit dashboard ensures that these complex backend operations remain

entirely transparent, explainable, and visually impressive.